

▼ Tuplas

Em programação, muitas vezes precisamos trabalhar com conjuntos de valores que pertencem ao mesmo contexto. Por exemplo, podemos querer representar as coordenadas de um ponto no plano, armazenar informações básicas de um aluno como nome, idade e matrícula, ou ainda representar as dimensões de um objeto como largura, altura e profundidade. Assim como acontece com as listas, criar uma variável separada para cada valor pode tornar o código menos organizado e mais difícil de entender.

Em Python, uma estrutura bastante utilizada para agrupar esse tipo de informação é a tupla. As tuplas permitem armazenar vários elementos dentro de uma única variável, mantendo uma ordem entre eles. Cada elemento possui uma posição dentro da sequência, e pode ser acessado por meio de um índice, da mesma forma que fazemos com listas.

Uma característica importante das tuplas é que elas são imutáveis. Isso significa que, depois de criadas, seus valores não podem ser modificados, removidos ou substituídos. Essa propriedade é útil em situações em que queremos garantir que determinados dados permaneçam constantes ao longo da execução do programa, evitando alterações acidentais.

As tuplas também podem armazenar diferentes tipos de dados ao mesmo tempo, como números, textos e até outras estruturas. Por isso, são frequentemente utilizadas para representar conjuntos fixos de informações relacionadas.

Nesta parte aula, iremos aprender o que são tuplas em Python, como criá-las e como acessar seus elementos. Também veremos em quais situações é mais interessante utilizar tuplas em vez de listas, e como elas podem ajudar a organizar melhor os dados dentro dos nossos programas.

```
tupla = (10, 20, 30, 40, 50)

print("Minha Tupla:", tupla)
print("Tipo:", type(tupla))
print("Elemento da Tupla", tupla[2])
```

```
Minha Tupla: (10, 20, 30, 40, 50)
Tipo: <class 'tuple'>
Elemento da Tupla 30
```

▼ Lista vs Tuplas

Embora listas e tuplas sejam estruturas semelhantes em Python, a principal diferença entre elas é que listas são mutáveis e tuplas são imutáveis. Isso significa que os elementos de uma lista podem ser alterados, adicionados ou removidos após sua criação, enquanto os valores de uma tupla permanecem fixos. Por esse motivo, listas são mais indicadas quando precisamos modificar os dados ao longo do programa, como em coleções que crescem ou mudam com o tempo. Já as tuplas são mais adequadas para representar conjuntos de informações que não devem ser alterados, como coordenadas, configurações ou dados constantes. Além disso, por serem imutáveis, tuplas costumam ser um pouco mais eficientes em termos de memória e desempenho.

```
lista = [10, 20, 30, 40, 50, 60]

print("Antes: ", lista)

lista[2] = 35

print("Depois:", lista)
```

```
Antes:  [10, 20, 30, 40, 50, 60]
Depois: [10, 20, 35, 40, 50, 60]
```

```
tupla = (10, 20, 30, 40, 50, 60)

print("Antes: ", tupla)

tupla[2] = 35

print("Depois:", tupla)
```

```

Antes: (10, 20, 30, 40, 50, 60)
-----
TypeError                                 Traceback (most recent call last)
Cell In[7], line 5
      1 tupla = (10, 20, 30, 40, 50, 60)
      3 print("Antes: ", tupla)
----> 5 tupla[2] = 35
      7 print("Depois:", tupla)

TypeError: 'tuple' object does not support item assignment

```

✓ Slicing

Em tuplas também podemos fazer o Slicing, igual os outros iteráveis que vimos (Strings e Listas).

```

# Acessando Elementos
tupla = (10, 20, 30, 40, 50)

# Acessando elemento por index
print(tupla[0])

# Acessando o ultimo elemento da tupla
print(tupla[-1])

# Acessando sub tupla dentro da tupla
print(tupla[2:4])

# Acessando os primeiros 2 elementos da tupla
print(tupla[:2])

# Acessando os elementos a partir do 3
print(tupla[3:])

# A partir dos 3 ultimos
print(tupla[-3:])

# Cortando os 3 ultimos
print(tupla[:-2])

10
50
(30, 40)
(10, 20)
(40, 50)
(30, 40, 50)
(10, 20, 30)

```

✓ Métodos da Tupla

Diferente das listas, as tuplas possuem poucos métodos, justamente porque são imutáveis. Como seus valores não podem ser alterados após a criação, métodos que adicionam, removem ou modificam elementos não existem para tuplas.

```

tupla = (10, 20, 10, 20, 10, 60)

print("Quantas vezes aparece 10? ", tupla.count(10), "vezes")
print("Qual a primeira ocorrência do 0? Index", tupla.index(20))

Quantas vezes aparece 10? 3 vezes
Qual a primeira ocorrência do 0? Index 1

```

O pertencimento de uma elemento a uma tupla também pode ser feito usando o `in` e `not in`

```

tupla = ("Zezé", "Becker", "Tim", "Toms", "Otávio", "Rafa")
print("Zezé" in tupla)

True

```

✓ Utilizando em For

Igual aos outros iteráveis que vimos, as tuplas também podem ser usada no `for`

```
t = (10, 20, 30, 40)

print("Ordem Normal: ")
for valor in t:
    print(valor)

print("\nOrdem reversa: ")
for v in reversed(t):
    print(v)
```

Ordem Normal:

10
20
30
40

Ordem reversa:

40
30
20
10

▼ Funções Adicionais

Assim como as listas podemos usar as funções de `len`, `sum`, `max` e `min`.

```
tupla = (10, 30, 20, 40, 50, 15)

print("Comprimento:", len(tupla))    # Quantos elementos a tupla tem
print("Soma:", sum(tupla))           # A soma de todos os elementos
print("Max:", max(tupla))            # O elemento de maior valor
print("Min:", min(tupla))            # O elemento de menor valor
```

Comprimento: 6
Soma: 165
Max: 50
Min: 10